JavaFX base con Maven

Guía paso a paso para crear, configurar y ejecutar un proyecto JavaFX

Programación Orientada a Objetos · DUOC UC
Proyecto: **1_javafx-base** · JavaFX 21 · JDK 21 · Maven 3.9

Índice

1. ¿Qué vamos a hacer y por qué?
2. Instalar las herramientas (JDK 21, Maven, IntelliJ)
3. Maven: qué es y cómo configurar un proyecto
4. Crear el proyecto JavaFX paso a paso
5. Entender el código: Application, Stage y Scene
6. Ejecutar por línea de comandos
7. Ejecutar con IntelliJ IDEA
8. Elementos que podemos usar (layouts y controles)
9. Los métodos más importantes
10. Ejemplo: una ventana con botón y evento
11. Errores frecuentes y cómo resolverlos
12. Glosario

1. ¿Qué vamos a hacer y por qué?

¿Qué es JavaFX?

JavaFX es una librería de Java para construir **aplicaciones con ventanas** (interfaz gráfica de usuario o GUI): botones, cajas de texto, tablas, etc. Hasta ahora hemos trabajado en la consola; con JavaFX el usuario interactúa con el programa usando el mouse y el teclado.

> **¿Para qué sirve en este ramo?** Nos permite aplicar la Programación Orientada a Objetos en algo visual: cada ventana es una clase, cada botón es un objeto, y cada clic es un evento que invoca a un método.

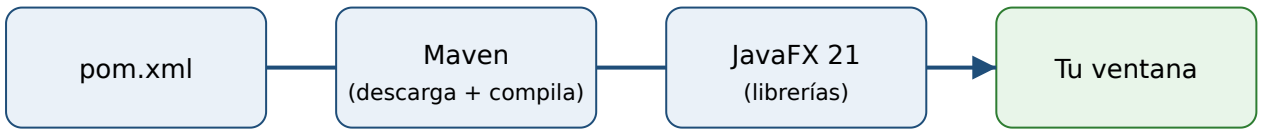
¿Qué es Maven y por qué lo usamos?

JavaFX **no viene incluido** en el JDK desde Java 11. Hay que descargarlo aparte. Maven es una herramienta que:

- **Descarga automáticamente** las librerías que necesitamos (dependencias), con la versión correcta.
- **Compila** el proyecto y lo **ejecuta** con un solo comando.
- Hace que el proyecto sea **igual en cualquier computador** (Linux, Windows, Mac) y en cualquier IDE.

> **Idea clave:** el archivo `pom.xml` describe el proyecto. Si tienes el `pom.xml`, Maven sabe todo lo necesario para construirlo. Por eso basta con copiar la carpeta y funciona.

Panorama general



2. Instalar las herramientas

Necesitamos tres cosas: **JDK 21** (para compilar y ejecutar Java), **Maven** (para construir) e **IntelliJ IDEA** (editor, opcional pero recomendado).

2.1 JDK 21

> **¿Por qué 21?** Es una versión LTS (soporte largo) y el proyecto está configurado con `release 21`. Si usas una versión menor, Maven mostrará un error de compilación.

Linux

Descarga el JDK 21 (por ejemplo Temurin, `adoptium.net`), descomprímelo y define las variables en `~/.bashrc`:

```
export JAVA_HOME="$HOME/tools/jdk-21"
export PATH="$JAVA_HOME/bin:$PATH"
```

Windows

1. Descarga el instalador `.msi` de Temurin 21 desde `adoptium.net` e instálalo (marca la opción *Set JAVA_HOME variable*).
2. Abre una **nueva** terminal (PowerShell) para que tome los cambios.

Verificar (ambos sistemas)

```
java -version
javac -version
```

Debe aparecer `21.x`.

2.2 Maven

Linux

```
# 1. Descargar y descomprimir (ejemplo en ~/tools)
curl -O https://archive.apache.org/dist/maven/maven-3/3.9.9/binaries/apache-maven-3.9.9-
bin.tar.gz
tar -xzf apache-maven-3.9.9-bin.tar.gz -C ~/tools

# 2. Agregar a ~/.bashrc
export PATH="$HOME/tools/apache-maven-3.9.9/bin:$PATH"
```

Windows

1. Descarga el `.zip` binario desde `maven.apache.org/download.cgi` y descomprímelo, por ejemplo en `C:\tools\apache-maven-3.9.9`.

2. Abre *Editar las variables de entorno del sistema* → *Variables de entorno* → en **Path** agrega `C:\tools\apache-maven-3.9.9\bin`.
3. Verifica que `JAVA_HOME` exista y apunte a la carpeta del JDK 21.

Verificar

```
mvn -v
```

Debe mostrar `Apache Maven 3.9.x` y la versión de Java 21.

> **Si usas solo IntelliJ** no es obligatorio instalar Maven: el IDE trae uno integrado. Pero para usar la línea de comandos sí lo necesitas.

2.3 IntelliJ IDEA

Descarga la edición **Community** (gratuita) desde `jetbrains.com/idea`. En Linux puedes usar el paquete `.tar.gz`; en Windows, el instalador `.exe`.

3. Maven: qué es y cómo configurar un proyecto

3.1 Estructura estándar de carpetas

Maven espera que los archivos estén en lugares fijos. Esto se llama *convención sobre configuración*: si respetas las carpetas, no hay que configurar nada más.

```
javafx-base/
├── pom.xml                    ← descripción del proyecto
└── src/
    ├── main/
    │   ├── java/              ← código fuente (.java)
    │   │   ├── module-info.java ← declara el módulo y qué necesita
    │   │   └── com/duoc/javafxbase/
    │   │       └── HelloApplication.java
    │   └── resources/         ← archivos no Java (fxml, imágenes, css)
    └── test/
        ├── java/              ← pruebas
        └── resources/
(target/ se crea al compilar: contiene los .class. No se edita ni se sube)
```

3.2 Crear un proyecto Maven desde cero (línea de comandos)

Maven puede generar la estructura por nosotros usando un *arquetipo* (una plantilla):

```
mvn archetype:generate \
    -DgroupId=com.duoc \
    -DartifactId=mi-proyecto \
    -DarchetypeArtifactId=maven-archetype-quickstart \
    -DarchetypeVersion=1.4 \
    -DinteractiveMode=false
```

En Windows PowerShell escribe el comando en **una sola línea** (sin las `\`).

Parámetro	Significado
`groupId`	Identifica a la organización o curso. Se escribe como dominio al revés (`com.duoc`). Suele ser el inicio del nombre del paquete.
`artifactId`	Nombre del proyecto (y de la carpeta que se crea).
`archetypeArtifactId`	La plantilla a usar. `maven-archetype-quickstart` es un proyecto Java simple.
`interactiveMode=false`	No preguntar nada por consola.

Ese proyecto es Java “normal”. Para convertirlo en JavaFX hay que agregar las dependencias y el plugin en el `pom.xml`, como veremos en la sección 4.

3.3 Crear un proyecto Maven desde IntelliJ

1. *File → New → Project…*
2. En el panel izquierdo elige **Java**. Build system: **Maven**.
3. JDK: selecciona **21** (si no aparece, *Add JDK…* o *Download JDK…*).
4. Despliega *Advanced Settings* y define **GroupId** = `com.duoc`, **ArtifactId** = nombre del proyecto.
5. *Create*. IntelliJ genera el `pom.xml` y las carpetas `src/main/java`.

3.4 Comandos Maven que debes conocer

Comando	¿Qué hace?
`mvn compile`	Compila el código de `src/main` y deja los `.class` en `target/`.
`mvn clean`	Borra la carpeta `target/`. Útil cuando algo queda “pegado”.
`mvn test`	Compila y ejecuta las pruebas.
`mvn package`	Empaqueta el proyecto (genera el `.jar` en `target/`).
`mvn javafx:run`	Compila y **ejecuta** la aplicación JavaFX (lo entrega el plugin de JavaFX).
`mvn clean compile`	Se pueden encadenar fases: primero limpia, luego compila.

4. Crear el proyecto JavaFX paso a paso

Paso 1 · Carpeta y estructura

Crea la carpeta del proyecto y las carpetas de Maven (o usa la sección 3.2 / 3.3):

```
mkdir -p javafx-base/src/main/java/com/duoc/javafxbase
mkdir -p javafx-base/src/main/resources
cd javafx-base
```

Paso 2 · El archivo pom.xml

Este es el `pom.xml` completo del proyecto. Debajo se explica cada parte.

pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <groupId>com.duoc</groupId>
    <artifactId>javafx-base</artifactId>
    <version>1.0-SNAPSHOT</version>
    <name>javafx-base</name>

    <properties>
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
        <maven.compiler.release>21</maven.compiler.release>
        <javafx.version>21.0.5</javafx.version>
        <main.class>com.duoc.javafxbase.HelloApplication</main.class>
        <main.module>com.duoc.javafxbase</main.module>
    </properties>

    <dependencies>
        <dependency>
            <groupId>org.openjfx</groupId>
            <artifactId>javafx-controls</artifactId>
            <version>${javafx.version}</version>
        </dependency>
    </dependencies>

    <build>
        <plugins>
            <plugin>
                <groupId>org.apache.maven.plugins</groupId>
                <artifactId>maven-compiler-plugin</artifactId>
                <version>3.13.0</version>
                <configuration>
                    <release>${maven.compiler.release}</release>
                </configuration>
            </plugin>
            <plugin>
```

```
                <groupId>org.openjfx</groupId>
                <artifactId>javafx-maven-plugin</artifactId>
                <version>0.0.8</version>
                <executions>
                    <execution>
                        <!-- Default-cli: permite ejecutar con "mvn javafx:run" -->
                        <id>default-cli</id>
                        <configuration>
                            <mainClass>${main.module}/${main.class}</mainClass>
                            <launcher>app</launcher>
                            <jlinkZipName>app</jlinkZipName>
                            <jlinkImageName>app</jlinkImageName>
                            <noManPages>true</noManPages>
                            <stripDebug>true</stripDebug>
                            <noHeaderFiles>true</noHeaderFiles>
                        </configuration>
                    </execution>
                </executions>
            </plugin>
        </plugins>
    </build>
</project>
```

Elemento	¿Para qué sirve?
`groupId / artifactId / version`	Identifican el proyecto. `-SNAPSHOT` indica que está en desarrollo.
`project.build.sourceEncoding`	Codificación UTF-8: evita problemas con tildes y la “ñ”.
`maven.compiler.release`	Versión de Java con la que se compila (21).
`javafx.version`	Versión de JavaFX. Se define una vez y se reutiliza con `${javafx.version}`.
`main.class / main.module`	Clase principal y nombre del módulo. Los usa el plugin para ejecutar.
`<dependencies>`	Librerías que descarga Maven. `javafx-controls` incluye botones, etiquetas, tablas y el núcleo de JavaFX (`javafx-base` y `javafx-graphics` llegan como dependencias).
`maven-compiler-plugin`	Plugin que compila el código con la versión de Java indicada.
`javafx-maven-plugin`	Plugin que agrega el comando `mvn javafx:run`. En `mainClass` se indica `módulo/clase`.

Paso 3 · module-info.java

src/main/java/module-info.java

```
module com.duoc.javafxbase {
    requires javafx.controls;

    exports com.duoc.javafxbase;
}
```

¿Por qué existe este archivo? Java 9+ organiza el código en *módulos*. Este archivo declara el nombre del módulo, qué otros módulos necesita (`requires`) y qué paquetes deja visibles (`exports`). JavaFX está dividido en módulos, y aquí decimos que necesitamos `javafx.controls` .

Importante: el nombre del módulo debe coincidir con `main.module` del `pom.xml` , y `module-info.java` va en la raíz de `src/main/java` (no dentro del paquete).

Paso 4 · La clase principal

src/main/java/com/duoc/javafxbase/HelloApplication.java

```
package com.duoc.javafxbase;

import javafx.application.Application;
import javafx.scene.Scene;
import javafx.scene.control.Label;
import javafx.stage.Stage;

public class HelloApplication extends Application {

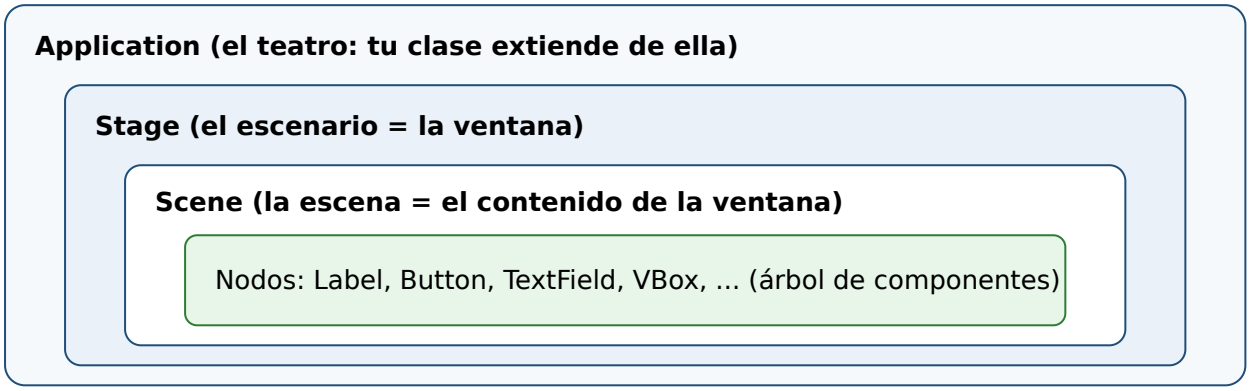
    @Override
    public void start(Stage stage) {
        Label label = new Label("Hola, JavaFX!");
        Scene scene = new Scene(label, 320, 240);

        stage.setTitle("javafx-base");
        stage.setScene(scene);
        stage.show();
    }

    public static void main(String[] args) {
        launch(args);
    }
}
```

5. Entender el código: Application, Stage y Scene

JavaFX usa la metáfora de un **teatro**:



Clase	Rol	En nuestro código
`Application`	Clase base de toda app JavaFX. Gestiona el ciclo de vida.	`extends Application`
`Stage`	La ventana. JavaFX te entrega la ventana principal en `start`.	parámetro `stage`
`Scene`	El contenido que se muestra en la ventana. Una ventana muestra una escena a la vez.	`new Scene(label, 320, 240)`
`Node`	Cualquier elemento visual (botón, etiqueta, contenedor...).	`Label`

Ciclo de vida

1. `main` llama a `launch(args)`: arranca JavaFX.
2. JavaFX crea un objeto de tu clase y llama a `init()` (opcional).
3. Llama a `start(Stage)`: **aquí construyes la interfaz**.
4. Al cerrar la ventana, llama a `stop()` (opcional): aquí se liberan recursos.

6. Ejecutar por línea de comandos

> **¿Por qué por consola?** Es la forma más confiable de saber si el proyecto funciona, sin depender del IDE. Si funciona aquí, funciona en cualquier computador con JDK 21 y Maven.

1. Abre una terminal (Linux) o PowerShell (Windows).
2. Entra a la carpeta del proyecto (donde está el `pom.xml`):

   ```
   cd 1_javafx-base
   ```

3. Comprueba las versiones:

   ```
   java -version
   mvn -v
   ```

4. Ejecuta la aplicación:

   ```
   mvn javafx:run
   ```

5. La primera vez Maven descarga JavaFX (puede tardar un poco). Luego se abre la ventana con el texto *“Hola, JavaFX!”*.
6. Para cerrar: cierra la ventana o presiona `Ctrl + C` en la terminal.

> **Solo compilar** (sin abrir la ventana): `mvn clean compile`. Un `BUILD SUCCESS` al final indica que todo está bien.

7. Ejecutar con IntelliJ IDEA

7.1 Abrir el proyecto

1. *File* → *Open...* y selecciona la carpeta del proyecto (o directamente el archivo `pom.xml` y elige *Open as Project*).
2. Si pregunta, confía en el proyecto (*Trust Project*).
3. Espera a que IntelliJ importe Maven (barra de progreso abajo a la derecha). Descarga JavaFX automáticamente.

7.2 Configurar el JDK

1. *File* → *Project Structure...* → *Project*.
2. En **SDK** selecciona el JDK 21 y en **Language level** “21”.
3. *Apply* → *OK*.

7.3 Ejecutar

Opción A (la más simple): abre `HelloApplication.java` y presiona el triángulo verde junto al método `main` → *Run 'HelloApplication.main()'*.

Opción B (igual que la consola): abre el panel **Maven** (a la derecha) → *Plugins* → *javafx* → *javafx:run* (doble clic).

> **Si aparece “JavaFX runtime components are missing”** al ejecutar con la Opción A: usa la Opción B, o revisa que el `pom.xml` haya sido importado (botón *Reload All Maven Projects* en el panel Maven).

8. Elementos que podemos usar

8.1 Contenedores (layouts)

Un layout organiza a sus hijos automáticamente. Casi nunca se posicionan elementos “a mano”.

Clase	¿Cómo ordena?	Cuándo usarlo
`VBox`	En columna, uno bajo otro.	Formularios simples, menús.
`HBox`	En fila, uno al lado del otro.	Botones alineados, barras.
`BorderPane`	5 zonas: top, bottom, left, right, center.	Estructura general de una ventana.
`GridPane`	En grilla (filas y columnas).	Formularios con etiqueta + campo.
`StackPane`	Uno encima de otro.	Centrar algo, superponer elementos.
`FlowPane`	Fluye y salta de línea al llenarse.	Galerías, etiquetas (tags).
`AnchorPane`	Anclado a los bordes con distancias.	Diseños con Scene Builder.
`ScrollPane`	Agrega barras de desplazamiento.	Contenido más grande que la ventana.

8.2 Controles

Control	Para qué	Método típico
`Label`	Mostrar texto.	`setText()`
`Button`	Ejecutar una acción.	`setOnAction()`
`TextField`	Texto de una línea.	`getText()`
`PasswordField`	Clave (oculta lo escrito).	`getText()`
`TextArea`	Texto de varias líneas.	`getText()`
`CheckBox`	Marcar sí/no.	`isSelected()`
`RadioButton` + `ToggleGroup`	Elegir una opción entre varias.	`isSelected()`
`ComboBox<T>`	Lista desplegable.	`getValue()`, `getItems()`
`ListView<T>`	Lista de elementos.	`getItems()`, `getSelectionModel()`
`TableView<T>`	Tabla de datos.	`getItems()`, `getColumns()`
`DatePicker`	Elegir una fecha.	`getValue()`
`Alert`	Mensajes emergentes (información, error, confirmar).	`showAndWait()`

Algunos controles (`Label`, `Button`, `TextField`...) están en el módulo `javafx.controls`, que ya incluimos. Para FXML se agrega `javafx.fxml` (ver proyecto 2).

9. Los métodos más importantes

Clase	Método	¿Qué hace?
`Application`	`launch(args)`	Inicia JavaFX. Se llama desde `main`.
`Application`	`start(Stage)`	Punto de entrada gráfico. Aquí se arma la interfaz.
`Application`	`init()` / `stop()`	Antes de abrir / al cerrar. Opcionales.
`Stage`	`setTitle(String)`	Título de la ventana.
`Stage`	`setScene(Scene)`	Define qué escena muestra la ventana.
`Stage`	`show()`	Muestra la ventana.
`Stage`	`setResizable(boolean)`	Permite o no cambiar el tamaño.
`Stage`	`close()`	Cierra la ventana.
`Scene`	`new Scene(raiz, ancho, alto)`	Crea la escena con un nodo raíz y tamaño.
`Scene`	`getWindow()`	Obtiene la ventana (Stage) de la escena.
`Node`	`setStyle(String)`	Aplica estilo CSS (`"-fx-text-fill: red;"`).
`Node`	`setDisable(boolean)`	Habilita o deshabilita un elemento.
`Pane / VBox / HBox`	`getChildren().add(nodo)`	Agrega un hijo al contenedor.
`VBox / HBox`	`setSpacing(double)`	Espacio entre hijos.
`VBox / HBox`	`setAlignment(Pos)`	Alineación de los hijos (`Pos.CENTER`).
`Region`	`setPadding(new Insets(10))`	Margen interior.
`Button`	`setOnAction(evento -> ...)`	Define qué hacer al hacer clic.
`Label`	`setText / getText`	Cambia o lee el texto.
`TextField`	`getText / setText / clear`	Lee, escribe o limpia el contenido.
`TextField`	`setPromptText(String)`	Texto de ayuda en gris.
`Alert`	`showAndWait()`	Muestra el mensaje y espera a que se cierre.

10. Ejemplo: ventana con botón y evento

Este ejemplo muestra cómo combinar layout, controles y un evento. Puedes reemplazar el método `start` de `HelloApplication` para probarlo (necesita `import javafx.scene.control.*; import javafx.scene.layout.VBox; import javafx.geometry.*;`).

```
@Override
public void start(Stage stage) {
    Label titulo = new Label("¿Cómo te llamas?");
    TextField nombreField = new TextField();
    nombreField.setPromptText("Escribe tu nombre");
    Button saludarBtn = new Button("Saludar");
    Label resultado = new Label();

    // Evento: se ejecuta cada vez que se hace clic
    saludarBtn.setOnAction(e -> {
        String nombre = nombreField.getText().trim();
        if (nombre.isEmpty()) {
            resultado.setText("Debes escribir un nombre.");
        } else {
            resultado.setText("¡Hola, " + nombre + "!");
        }
    });

    VBox raiz = new VBox(10, titulo, nombreField, saludarBtn, resultado);
    raiz.setAlignment(Pos.CENTER);
    raiz.setPadding(new Insets(20));

    stage.setTitle("Saludo");
    stage.setScene(new Scene(raiz, 320, 240));
    stage.show();
}
```

Qué demuestra: (1) los componentes son *objetos* que creamos con `new`; (2) el `VBox` los organiza; (3) `setOnAction` recibe un método (lambda) que se ejecuta cuando ocurre el evento. Es programación orientada a eventos.

11. Errores frecuentes y cómo resolverlos

Mensaje / síntoma	Causa	Solución
`mvn: command not found` / "no se reconoce el comando"	Maven no está en el PATH.	Revisar la sección 2.2 y abrir una terminal nueva.
`invalid target release: 21` / `release version 21 not supported`	Maven está usando un JDK menor a 21.	Revisar `mvn -v` y `JAVA_HOME`. En IntelliJ, cambiar el JDK del proyecto.
`Error: JavaFX runtime components are missing`	Se ejecutó sin las librerías de JavaFX en el module path.	Ejecutar con `mvn javafx:run`.
`Module not found: javafx.controls`	Falta la dependencia o Maven no se importó.	Revisar el `pom.xml` y recargar Maven.
`Package ... does not exist` en `module-info`	El nombre del paquete en `exports` no existe.	Debe coincidir exactamente con la carpeta del paquete.
Se abre la ventana pero sale vacía o no cambia	Quedó código compilado antiguo.	`mvn clean compile` y volver a ejecutar.
Aparecen símbolos raros en vez de tildes	Codificación incorrecta.	Guardar archivos en UTF-8 (ya está configurado en el `pom.xml`).

12. Glosario

Término	Significado
GUI	Interfaz gráfica de usuario (ventanas, botones, etc.).
Maven	Herramienta que gestiona dependencias, compilación y ejecución.
pom.xml	Archivo de configuración de Maven (Project Object Model).
Dependencia	Librería externa que el proyecto necesita.
Plugin	Extensión de Maven que agrega tareas (compilar, ejecutar).
Módulo	Agrupación de paquetes con reglas explícitas de qué usa y qué expone.
Stage	La ventana.
Scene	El contenido de la ventana.
Node	Cualquier elemento visual de la escena.
Layout	Contenedor que ordena a sus hijos.
Evento	Algo que ocurre (clic, tecla) y a lo que el programa responde.
Lambda	Forma corta de escribir un método anónimo: `e -> { ... }`.

Siguiente paso: proyecto **2_javafx-fxml** (vistas con FXML y Scene Builder).